

Fundamentos de la programación

Laboratorio. Hoja 4

Los ejercicios marcados como **Evaluable** tienen que subirse al CV durante la sesión de laboratorio.

- Se subirá **únicamente** el archivo **Program.cs**
- Sólo hará la entrega uno de los miembros del grupo.
- La línea 1 (y siguientes) deben contener los nombres de los alumnos que hacen la entrega de la forma:

```
// Nombre Apellido1 Apellido2  
// Nombre Apellido1 Apellido2
```

-
1. (**Evaluable**) Implementar la clase **Rational** para representar y manipular números racionales. Dicha clase tendrá dos atributos **num** y **den** de tipo entero para representar el numerador y el denominador. Consideramos los siguientes invariantes de representación:
 - el denominador siempre será positivo (el signo solo se aplica en el numerador y no se admite denominador nulo).
 - la fracción $\frac{\text{num}}{\text{den}}$ debe ser *irreducible*. Para ello se implementará un método privado **Normalize** divida el numerador y denominador entre su *mcd* (se implementará un método privado para calcular el mcd de dos números enteros).

Nótese que estos invariantes garantizan la unicidad de representación para cualquier número racional.

La clase debe implementar métodos para leer y escribir números racionales.

2. Extender la clase anterior para incluir las operaciones de acceso al numerador y al denominador (*getter* y *setter*) mediante propiedades (tal como hemos visto en clase para los complejos). Además se implementarán las operaciones habituales: suma, resta, multiplicación, división, mediante sobrecarga de los operadores correspondientes.

Implementar también la conversión a string (método **toString**) para escritura, método (**Parse**) para lectura, método **ToDouble** para conversión a *double*. Extender el método *Parse* para que sea también capaz de convertir un número en coma flotante a racional (con la consiguiente pérdida de precisión, obviamente).

Las operaciones susceptibles de producir error deben lanzar el mensaje de error adecuado.

Implementar un método **Main** para probar la clase.

3. Implementar una clase **Coordenada** para representar puntos en el plano cartesiano y permita realizar las operaciones básicas con ellos.

La clase tendrá dos atributos privados **x**, y de tipo *double* que representen la posición en el plano, una constructora que reciba los valores iniciales de **x,y** y otro sin argumentos que los inicialice a 0. Tendrá además métodos *getter* y *setter* para acceder y modificar los valores de los atributos.

Además se implementarán los siguientes métodos públicos:

- `double DistanciaAlOrigen():` Devuelve la distancia desde el punto al origen (0,0).
- `double DistanciaA(Coordenada otra):` Devuelve la distancia entre la coordenada actual y otra coordenada recibida como parámetro.
- `void Mover(double dx, double dy):` Modifica la posición de la coordenada sumándole el vector (dx, dy).
- `string ToString():` Devuelve una cadena con la representación de la coordenada en formato "(x, y)".

Crear una clase Programa con un Main() que pruebe la funcionalidad de la clase Coordenada: crear un par de coordenadas y mostrar su distancia al origen. Mover una coordenada y mostrar el resultado. Calcular la distancia entre dos coordenadas y mostrarla en pantalla.